

ESPECIFICACIÓN TÉCNICA – Sistema Federativo de Historial y Medallero (MVP)

0) Objetivo del sistema

Construir un sistema web oficial de federación para registrar competencias, resultados, y mostrar el historial de atletas (incluyendo independientes) y clubes nacionales. El medallero es una vista derivada y auditable. El sistema no es un sistema de entrenamiento ni un reemplazo de IANSEO.

1) Alcance del MVP (cerrado)

Incluye: CRUD de Atletas, Clubes, Eventos y Categorías. Definir qué se compite en un evento (Evento + Categoría + Modalidad). Carga de resultados individuales por fase (Qualification y Final). Medallero Nacional de Clubes (solo eventos habilitados). Perfiles de Atleta y Club con historial. Excluye (futuro): Flecha por flecha, tandas y rondas detalladas. Importación automática IANSEO. Entrenamientos. Configuración avanzada de arco. Resultados por equipos o mixtos.

2) Stack recomendado

Frontend: React + Vite, Bootstrap, React Router, Axios/Fetch.

Backend: Node.js (TypeScript), Express, Prisma ORM, JWT, Zod, Helmet, CORS, Rate Limiting.

Base de datos: PostgreSQL administrado por Supabase.

Almacenamiento de imágenes: Supabase Storage.

3) Arquitectura

Arquitectura monorepo: /apps/api – Backend Express /apps/web – Frontend React Patrón backend: routes → controllers → services → prisma repository, con middlewares de autenticación, roles, validación y manejo de errores.

4) Reglas de negocio oficiales

Atletas independientes: clubId puede ser NULL, aparecen en historial personal pero no suman al medallero de clubes.

Medallas: no se almacenan como texto. Se derivan de la fase y posición.

FINAL + posición 1 = Oro; FINAL + posición 2 = Plata; BRONZE_MATCH ganador = Bronce.

Ponderación: calculada dinámicamente (no persistida).

Eventos para medallero de clubes: clubMedalsEnabled = true y technicalLevel WA_STANDARD o INDOOR_STANDARD.

5) Modelo de datos (Prisma – resumen)

Entidades principales: Club (con logoUrl) Athlete (con photoUrl) Event Category Modality Phase EventCategory Result User Las URLs de imágenes se almacenan como strings en PostgreSQL; los archivos viven en Supabase Storage.

6) Imágenes (Supabase Storage)

Buckets públicos: athletes/ → fotos de atletas clubs/ → logos de clubes Flujo: Admin sube imagen → Backend la envía a Supabase Storage → se obtiene URL pública → se guarda en la base de datos.

7) API REST (resumen)

Auth, Clubs, Athletes, Categories, Events, EventCategories, Results, Medallero calculado y Perfiles (Atleta y Club). Endpoints protegidos con JWT y roles ADMIN / SUPER_ADMIN.

8) Seguridad mínima

Hash de contraseñas con bcrypt, JWT con expiración, RBAC, CORS restringido, Helmet y rate limiting en login.

9) Despliegue

Frontend: Vercel o Netlify.

Backend: Render o Railway.

Base de datos y Storage: Supabase (free tier).

10) Uso con Copilot / Agente

Este documento debe entregarse a Copilot o a un agente autónomo como especificación oficial. El agente debe implementar primero la base de datos y API, luego el frontend, siguiendo estrictamente estas reglas.